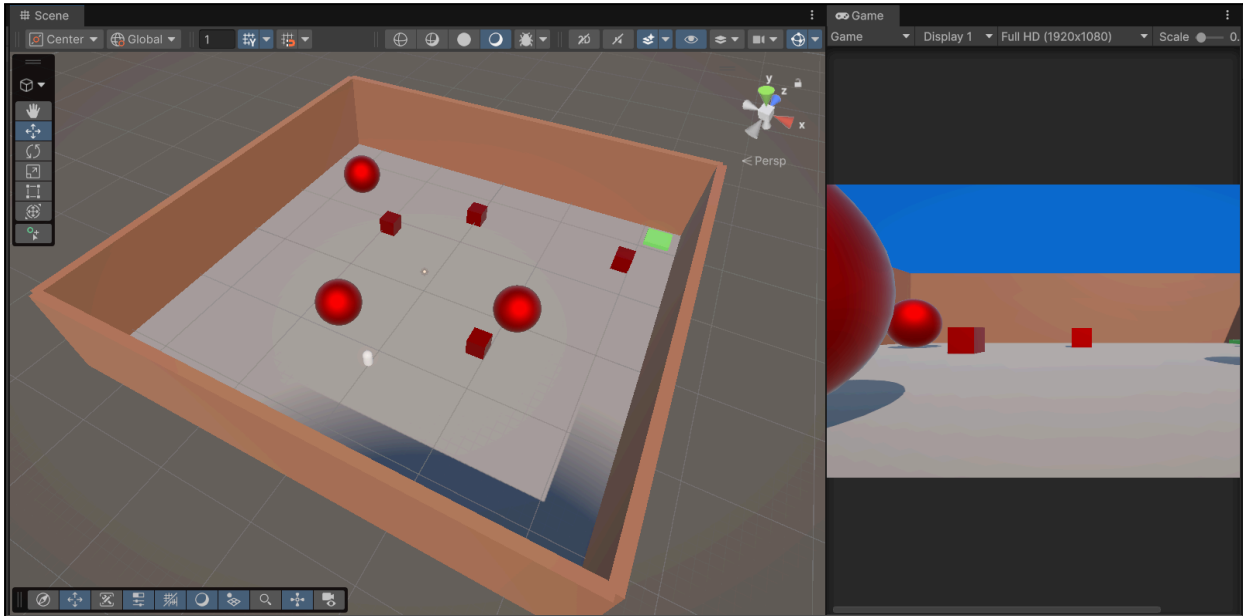


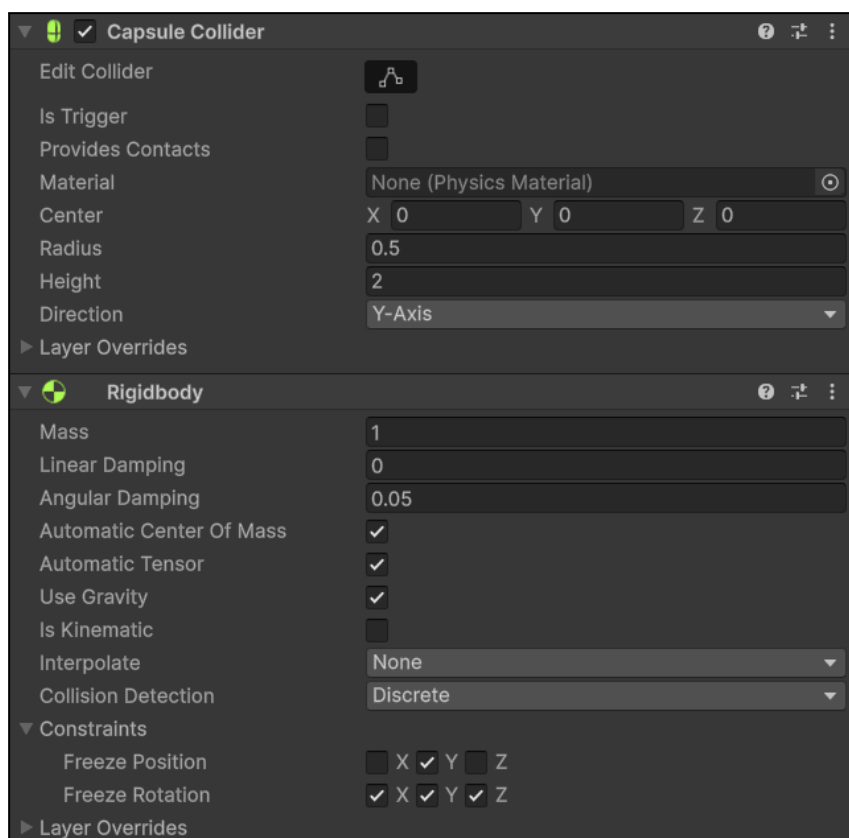
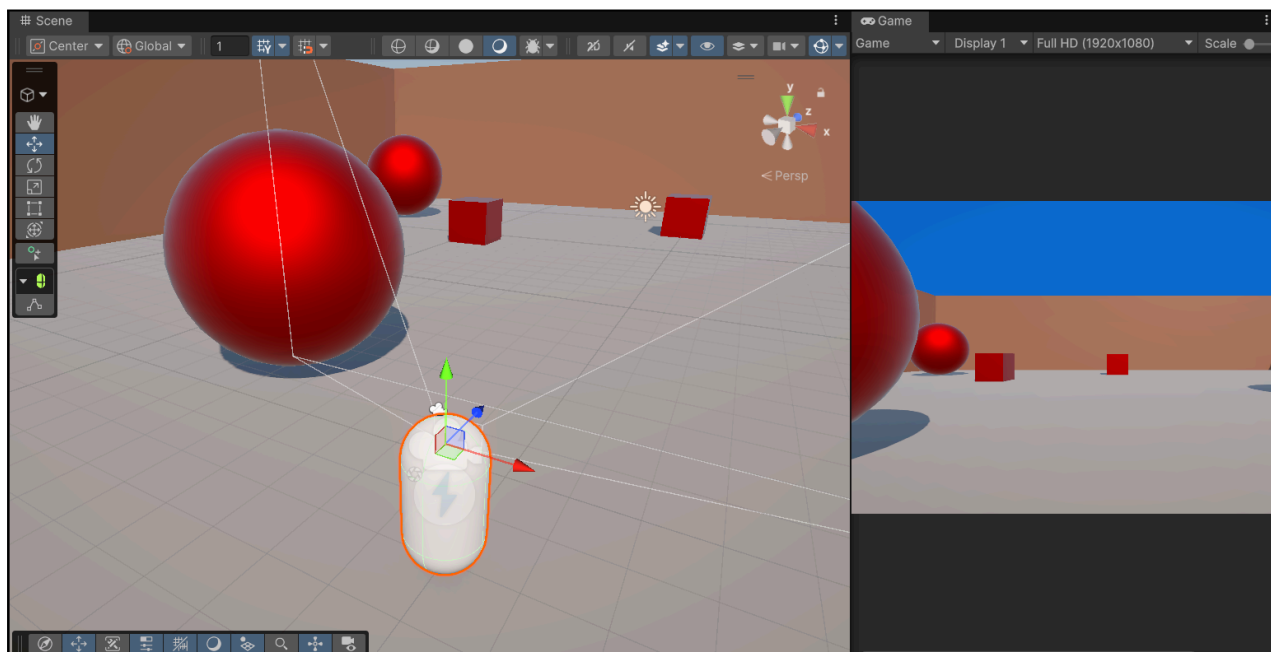
	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO 05

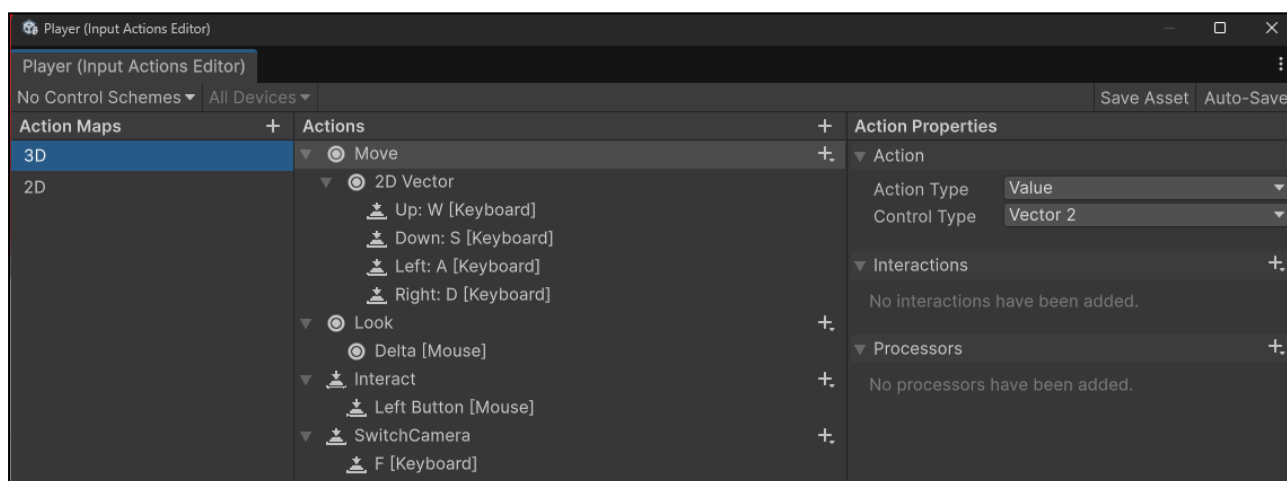
INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Proyección y Perspectiva</i>				
NÚMERO DE PRÁCTICA:	05	AÑO LECTIVO:	2026	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	26/05/26	HORA DE PRESENTACIÓN			
ESTUDIANTE:	Ponce de Leon Aguilar, Marco Antonio			NOTA:	
DOCENTE:	<i>Rene Alonso Nieto Valencia</i>				

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Crear un proyecto con tres escenas enfocadas en la proyección y la perspectiva. 1. Escena 1: Modificación de tamaño de objetos según la perspectiva del jugador Se creó un escenario con diferentes objetos: cubos y esferas, con las cuales el jugador puede interactuar. También está la zona verde para pasar a la siguiente escena. 

El jugador es una cápsula con su respectivo collider y rigidbody, con los bloqueos respectivos para el movimiento en 3D.



Se creó un archivo de InputActions para los controles del jugador.



Y en el código, el movimiento del jugador está gestionado por **PlayerController**.

```

1 public class PlayerController : MonoBehaviour {
2     [Header("Movement")]
3     [SerializeField] private float speed = 10f;
4
5     [Header("Camera")]
6     [SerializeField] public Transform cameraRoot;
7     [SerializeField] private float mouseSensitivity = 0.2f;
8     [SerializeField] private float verticalClamp = 80f;
9
10    private Rigidbody rb;
11    private ObjectPickup objectPickup;
12    private Vector2 movementVector;
13    private Vector2 lookVector;
14    private float verticalRotation = 0f;
15
16    private void Awake() {
17        rb = GetComponent<Rigidbody>();
18        objectPickup = GetComponent<ObjectPickup>();
19        Cursor.lockState = CursorLockMode.Locked;
20        Cursor.visible = false;
21    }
22
23    public void OnMove(InputValue value) {
24        movementVector = value.Get<Vector2>();
25    }

```

```
27     public void OnLook(InputValue value) {
28         lookVector = value.Get<Vector2>();
29     }
30
31     public void OnInteract(InputValue value) {
32         objectPickup.TryInteract();
33     }
34
35     private void FixedUpdate() {
36         HandleMovement();
37     }
38
39     private void Update() {
40         HandleLook();
41     }
42
43     private void HandleMovement() {
44         Vector3 direction = GetCameraRelativeDirection();
45         Vector3 newLinearVelocity = direction * speed;
46         newLinearVelocity.y = rb.linearVelocity.y;
47         rb.linearVelocity = newLinearVelocity;
48     }
49
50     private void HandleLook() {
51         transform.Rotate(lookVector.x * mouseSensitivity * Vector3.up);
52
53         verticalRotation -= lookVector.y * mouseSensitivity;
54         verticalRotation = Mathf.Clamp(verticalRotation, -verticalClamp, verticalClamp);
55         cameraRoot.localRotation = Quaternion.Euler(verticalRotation, 0f, 0f);
56     }
57
58     private Vector3 GetCameraRelativeDirection() {
59         Vector3 cameraForward = Vector3.ProjectOnPlane(cameraRoot.forward,
60             Vector3.up).normalized;
61         Vector3 cameraRight = Vector3.ProjectOnPlane(cameraRoot.right,
62             Vector3.up).normalized;
63
64         return cameraForward * movementVector.y + cameraRight * movementVector.x;
65     }
66 }
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

Cuando se presiona el click del mouse, se llama a la función **OnInteract**, como se definió en el mapa de acciones previamente.

```

31     public void OnInteract(InputValue value) {
32         objectPickup.TryInteract();
33     }

```

El componente **objectPickup** funciona con el siguiente código, permitiendo acercar y alejar los objetos dando la impresión de que mantienen su tamaño original, para que al dejar de interactuar, se pueda notar el cambio. Este componente modifica el uso de gravedad del rigidbody con el objeto con el cual se esté interactuando.

```

1  public class ObjectPickup : MonoBehaviour {
2      [Header("Pickup Settings")]
3      [SerializeField] private Transform cameraRoot;
4      [SerializeField] private float holdDistance = 8f;
5      [SerializeField] private float pickupRange = 8f;
6      [SerializeField] private LayerMask pickupLayer;
7
8      private GameObject heldObject;
9      private float originalDistance;
10     private Vector3 originalScale;
11     private Vector3 holdOffset;
12
13     private void Update() {
14         if (heldObject != null) HoldObject();
15     }
16
17     private void HoldObject() {
18         heldObject.transform.position = cameraRoot.position + cameraRoot.forward *
holdDistance + holdOffset;
19     }
20
21     public void TryInteract() {
22         if (heldObject == null) TryPickup();
23         else DropObject();
24     }

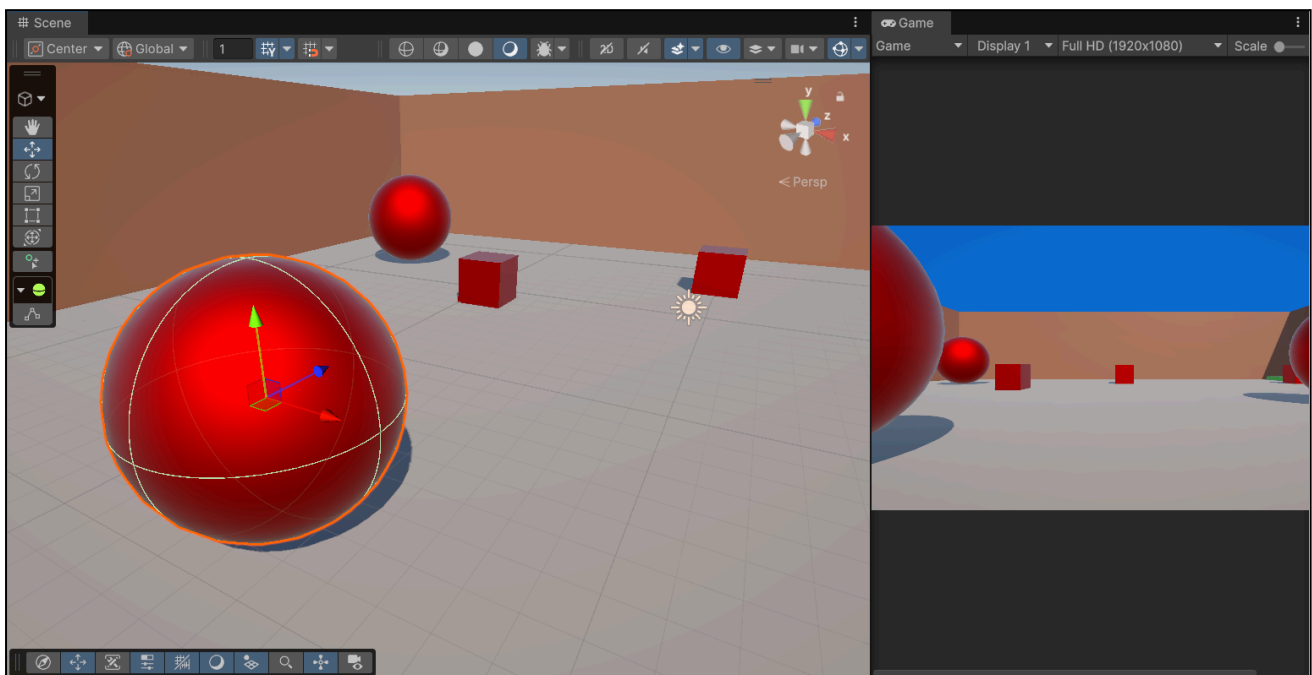
```

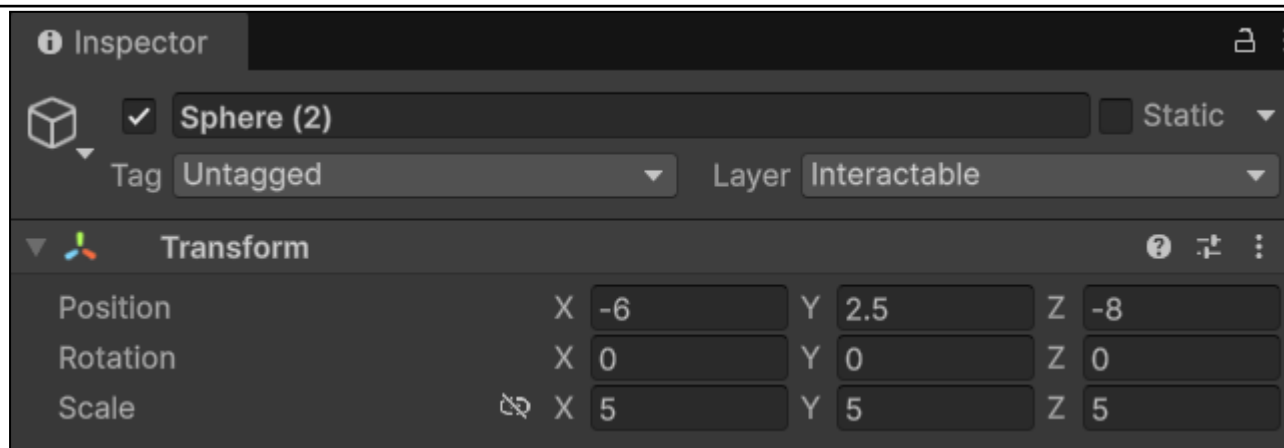
```

26     private void TryPickup() {
27         Ray ray = new Ray(cameraRoot.position, cameraRoot.forward);
28         if (Physics.Raycast(ray, out RaycastHit hit, pickupRange, pickupLayer)) {
29             heldObject = hit.collider.gameObject;
30             originalDistance = hit.distance;
31             originalScale = heldObject.transform.localScale;
32             holdOffset = heldObject.transform.position - (cameraRoot.position +
heldObject.transform.position - (cameraRoot.position +
cameraRoot.forward * holdDistance);
33
34             Rigidbody heldObjectRb = heldObject.GetComponent<Rigidbody>();
35             if (heldObjectRb != null) heldObjectRb.isKinematic = true;
36         }
37     }
38
39     private void DropObject() {
40         float scaleFactor = holdDistance / originalDistance;
41         heldObject.transform.localScale = originalScale * scaleFactor;
42
43         Rigidbody heldObjectRb = heldObject.GetComponent<Rigidbody>();
44         if (heldObjectRb != null) heldObjectRb.isKinematic = false;
45         heldObject = null;
46     }
47 }

```

Los objetos interactuables están en la capa **Interactable** para poder interactuar únicamente con ellos y no con cualquier otro objeto del entorno. El Raycast solamente busca dicha capa.

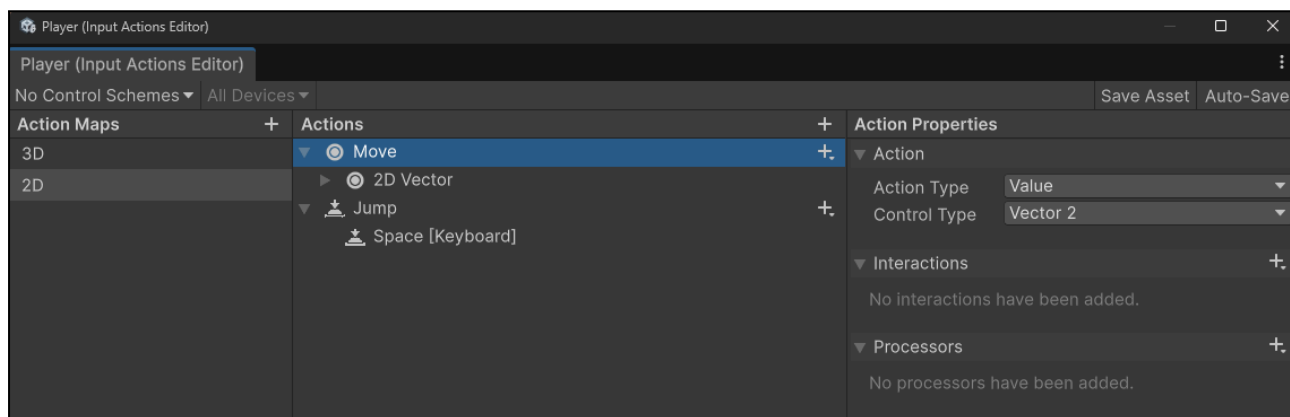




2. Escena 2: Combinación de dimensiones 2D y 3D

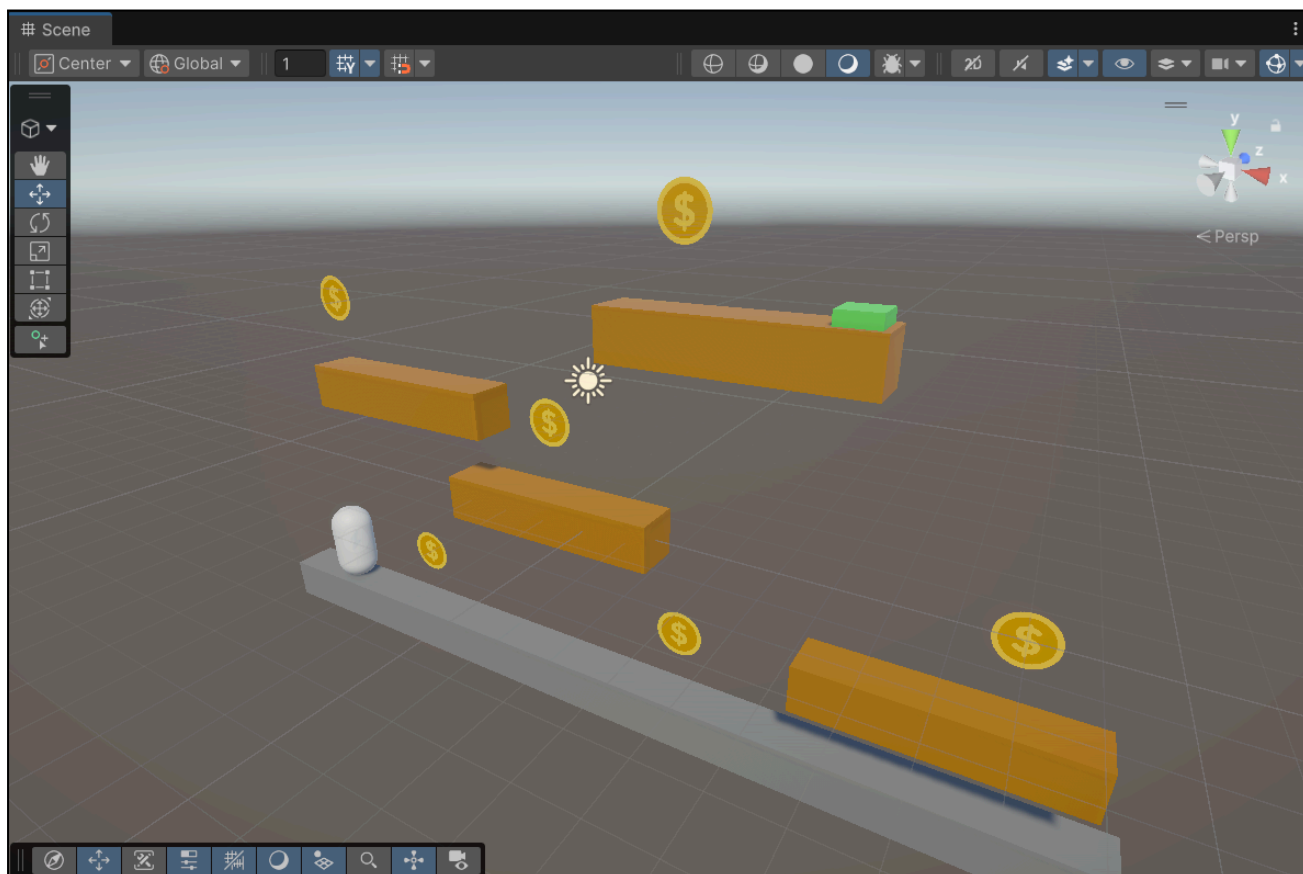
En esta escena se creó un plataformero en el cual el jugador **3D** puede moverse y saltar, recolectando monedas en **2D**.

En el mapa de acciones se agregó un nuevo grupo de acciones exclusivamente para **2D**. Gracias a ello, ahora el jugador no realiza todas las acciones que hacía previamente en **3D**.



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

Se creó el siguiente escenario con monedas en **2D** y la respectiva zona verde para pasar a la siguiente escena.



El código de movimiento de jugador ahora con salto, es el siguiente, considerando para el desplazamiento únicamente el **eje X**.

```

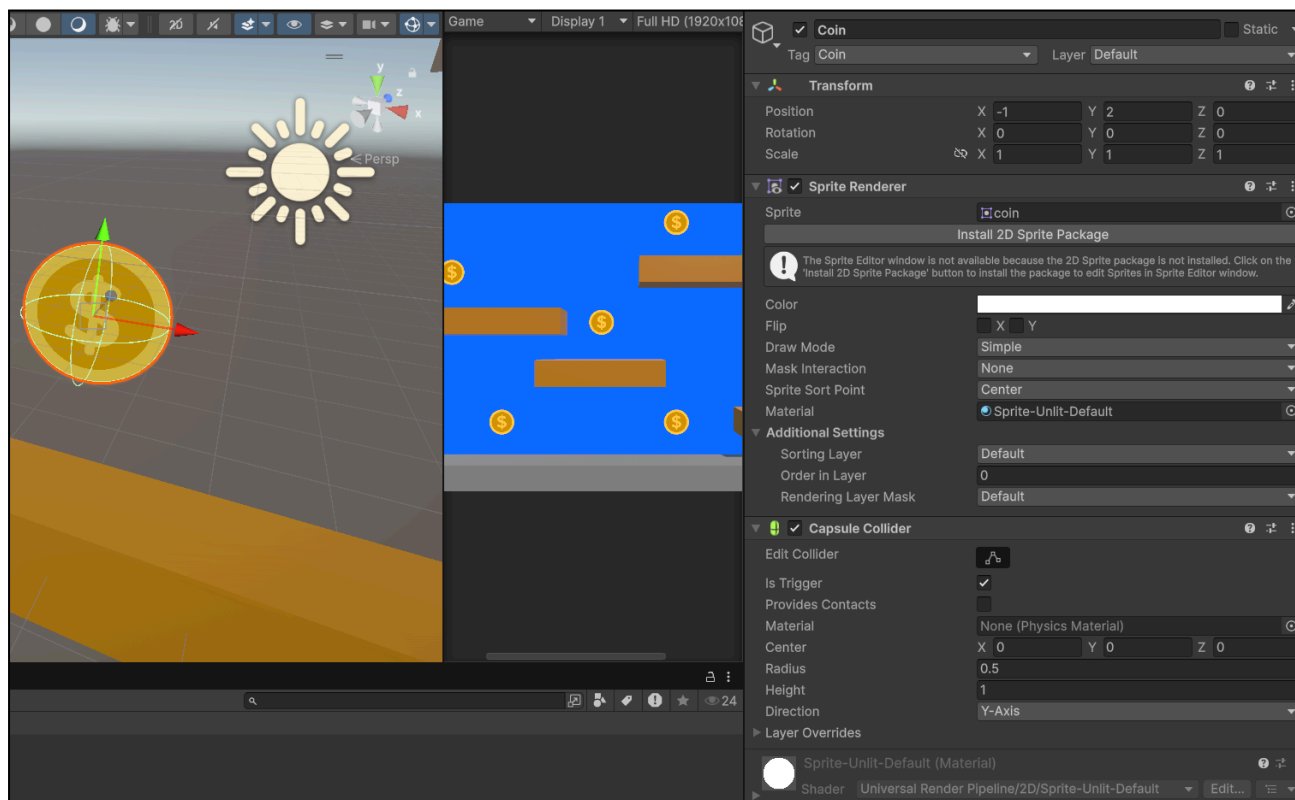
1  public class PlatformerPlayerController : MonoBehaviour {
2      [Header("Movement")]
3      [SerializeField] private float speed = 5f;
4      [SerializeField] private float jumpForce = 7f;
5
6      private Rigidbody rb;
7      private float movement;
8      private bool jumpRequested;
9      private bool isGrounded;
10
11     private void Awake() {
12         rb = GetComponent<Rigidbody>();
13     }

```



```
15     public void OnMove(InputValue value) {
16         movement = value.Get<Vector2>().x;
17     }
18
19     public void OnJump(InputValue value) {
20         if (value.isPressed && isGrounded) {
21             jumpRequested = true;
22         }
23     }
24
25     private void FixedUpdate() {
26         HandleMovement();
27         HandleJump();
28     }
29
30     private void HandleMovement() {
31         Vector3 velocity = rb.linearVelocity;
32         velocity.x = movement * speed;
33         rb.linearVelocity = velocity;
34     }
35
36     private void HandleJump() {
37         if (!jumpRequested) return;
38
39         jumpRequested = false;
40
41         rb.AddForce(Vector3.up * jumpForce, ForceMode.Impulse);
42     }
43
44     private void OnCollisionStay(Collision collision) {
45         isGrounded = true;
46     }
47
48     private void OnCollisionExit(Collision collision) {
49         isGrounded = false;
50     }
51 }
```

Las monedas cuentan con la etiqueta **Coin** para detectar la colisión, además de contar con la opción **Is Trigger** activa en su collider.



Y el jugador detecta la colisión con la siguiente función, que también detecta la colisión con la zona verde, la cual cuenta con la etiqueta **TP**.

```

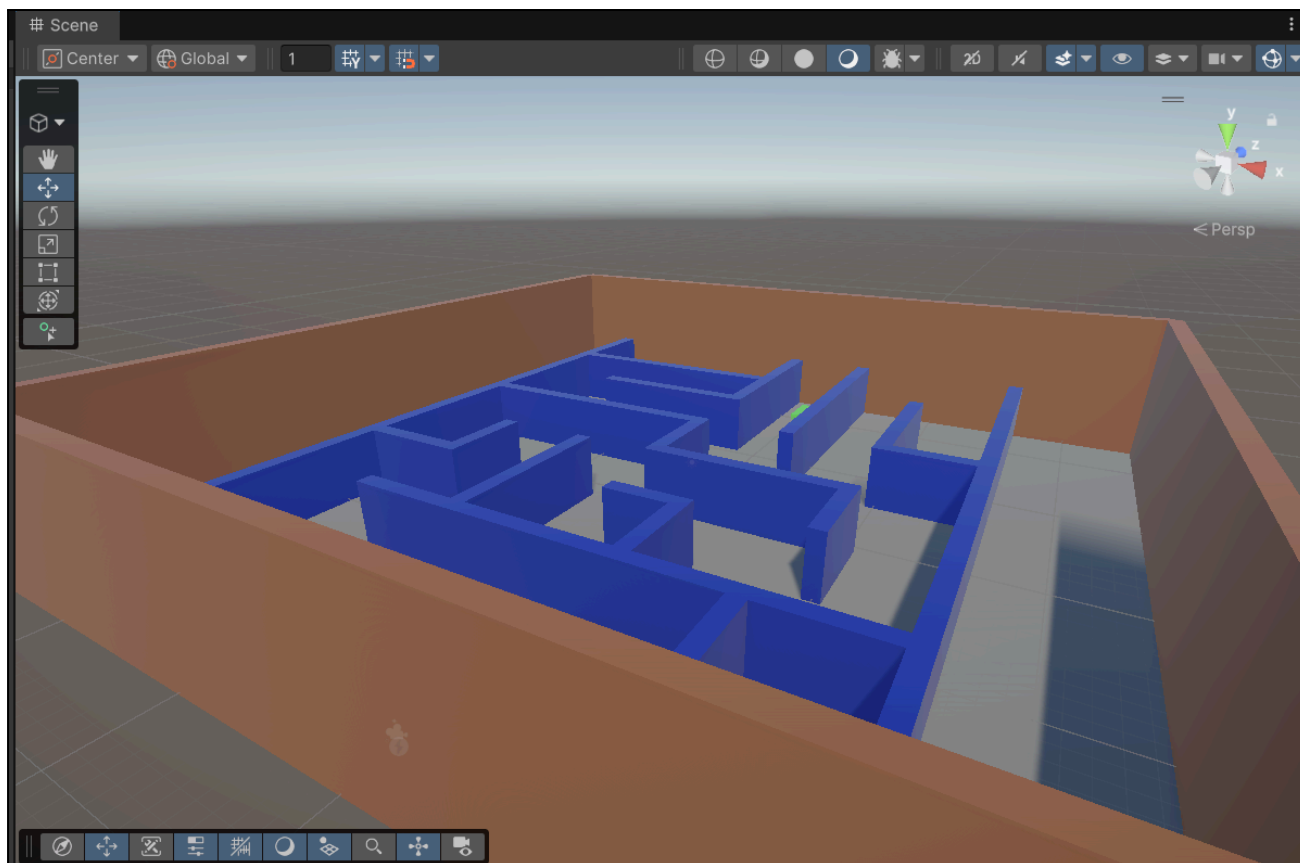
1 public class CollisionController : MonoBehaviour {
2     private void OnTriggerEnter(Collider collider) {
3         string colliderTag = collider.tag;
4         switch (colliderTag) {
5             case "TP":
6                 LevelManager.LoadNextLevel();
7                 break;
8             case "Coin":
9                 Destroy(collider.gameObject);
10                break;
11        }
12    }
13 }

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

3. Escena 3: Juego con mecánica de proyecciones y perspectiva

Para esta escena se creó un laberinto, el cual en un principio es difícil de reconocer el camino, por lo cual se recurre a un cambio en la perspectiva de la cámara para poder ubicarse con mayor facilidad.



Se reutiliza el **Player** de la primera escena, con la diferencia de que ahora se detecta cuando se presiona la acción **OnSwitchCamera**.

```

1 public void OnSwitchCamera(InputValue value) {
2     SwitchCameraValues();
3 }

```

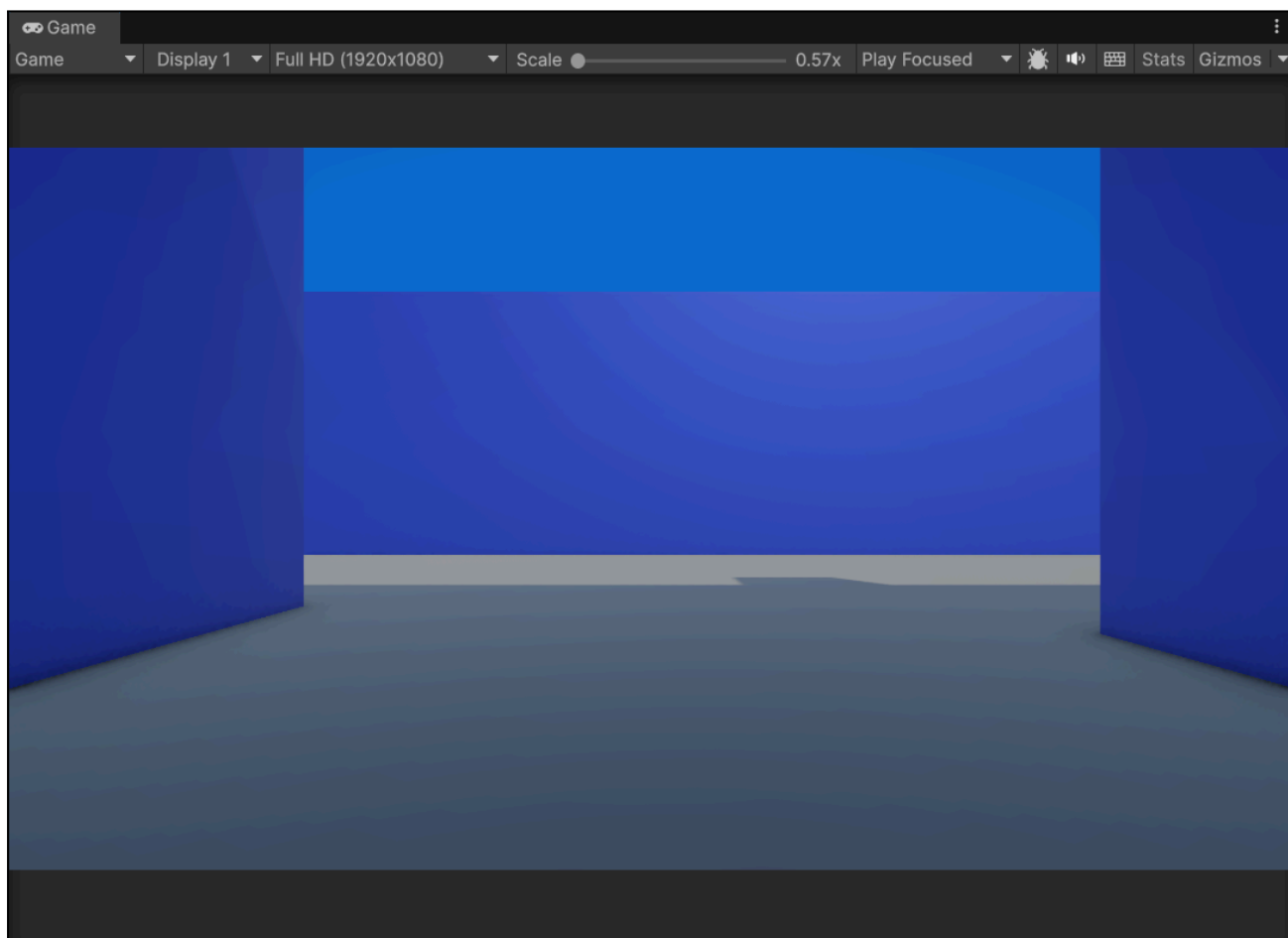
Y la siguiente función se encarga de modificar la perspectiva de la cámara.

```

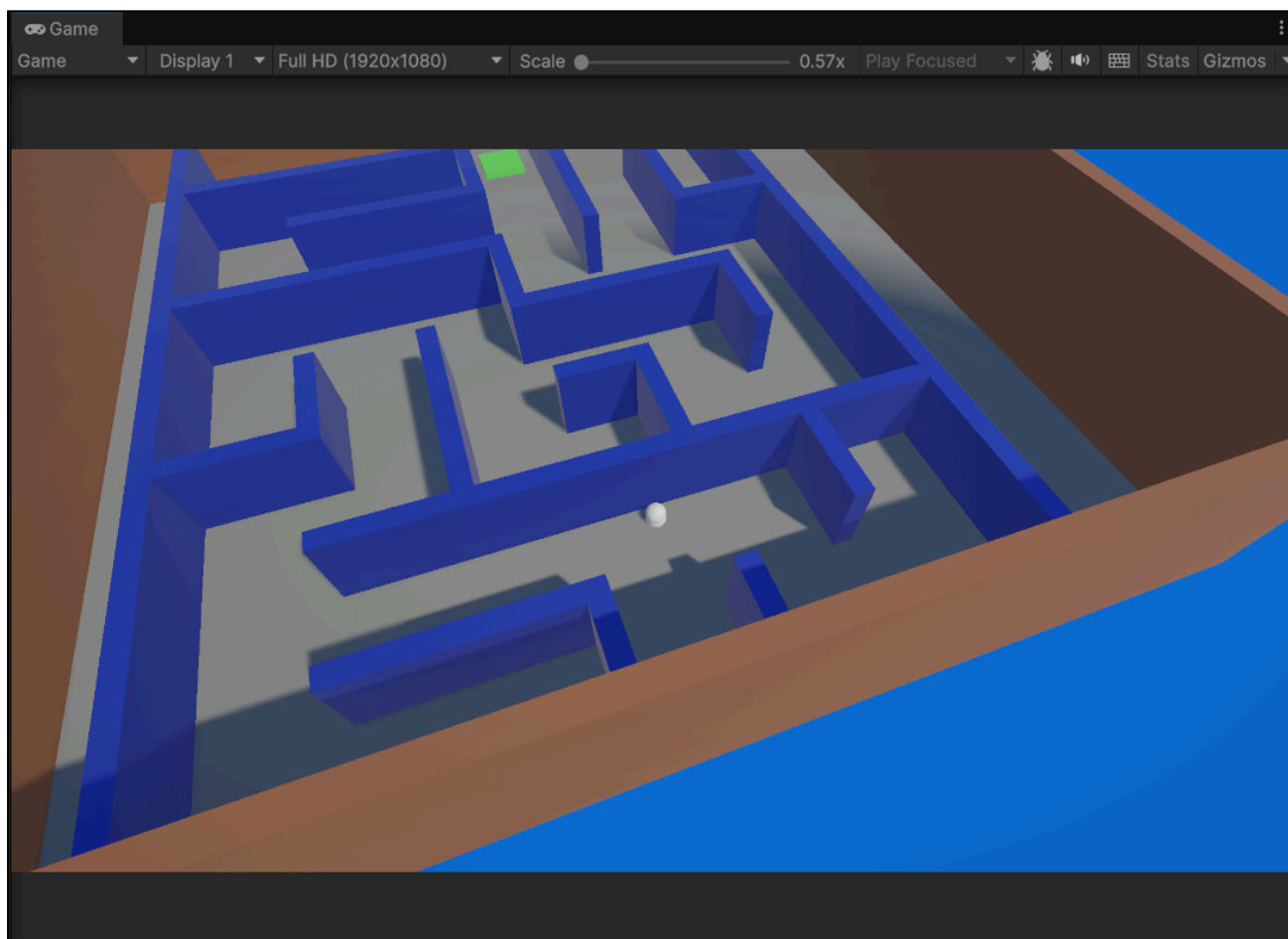
1 private void SwitchCameraValues() {
2     Vector3 cameraLocalPosition = mainCamera.transform.localPosition;
3     if (cameraValuesAreOnDefault) {
4         cameraLocalPosition.z = -30f;
5         cameraValuesAreOnDefault = false;
6     }
7     else {
8         cameraLocalPosition.z = 0f;
9         cameraValuesAreOnDefault = true;
10    }
11    mainCamera.transform.localPosition = cameraLocalPosition;
12 }

```

Originalmente el jugador cuenta con una cámara en primera persona:



Y gracias al cambio de perspectiva de la cámara, ahora puede ubicar de mejor forma dónde está:



4. Enlace al video de demostración

<https://drive.google.com/file/d/1tWAbaj6ltRYDuN9du9d7HGSLMxbrw6ES/view?usp=sharing>

CUESTIONARIO

1. ¿Qué problemas pueden presentarse al utilizar múltiples proyecciones en una escena?

El uso de múltiples proyecciones puede generar inconsistencias en la percepción de profundidad, tamaño y distancia de los objetos. También puede aumentar la complejidad del diseño de la interfaz y afectar la experiencia del usuario si los cambios de perspectiva no son claros o intuitivos.

2. ¿Cómo han realizado la simulación de una escena 3D con elementos 2D?

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

Se utilizó un entorno y un personaje en 3D, mientras que algunos elementos interactivos, como las monedas, se representaron mediante sprites 2D. Para la interacción se emplearon colliders y física 3D, permitiendo integrar objetos visualmente 2D dentro de una escena tridimensional.

RETROALIMENTACIÓN GENERAL